Experiment-8

Aim: Demonstrate the Dining Philosophers Synchronization Problem.

Description

The dining philosophers problem demonstrates how concurrent processes compete for shared resources. In this program, each philosopher is represented by a thread and each chopstick by a semaphore initialized to 1. A philosopher thinks, acquires two adjacent chopsticks, eats, and then releases both chopsticks. The number of philosophers is entered by the user, and each thread performs this sequence once. Since every philosopher acquires chopsticks in the same order, the program can deadlock if all philosophers hold one chopstick and wait for the next. It illustrates resource synchronization but does not guarantee freedom from deadlock or starvation.

Algorithm

1. Start and read the number of philosophers, n.
2. Initialize one semaphore for each chopstick with a value of 1.
3. Assign each philosopher an ID from 0 to n - 1 and create a thread for each.
4. In each thread, display the thinking message and wait for one second.
5. Acquire chopstick[id], then chopstick[(id + 1) % n], using sem_wait().
6. Display the eating message and wait for two seconds.
7. Release both chopsticks using sem_post(), display the completion message, and end the thread.
8. Wait for all philosopher threads to finish using pthread_join().
9. Destroy the chopstick semaphores and terminate the program.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t chopstick[50];
int n;
void* philosopher(void* arg){
    int id = *(int*)arg;
    printf("Philosopher %d is thinking\n", id);
    sleep(1);
    sem_wait(&chopstick[id]);
    sem_wait(&chopstick[(id + 1) % n]);
    printf("Philosopher %d is eating\n", id);
    sleep(2);
    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id + 1) % n]);
    printf("Philosopher %d finished eating\n", id);
    return NULL;
}
int main(){
    printf("Enter the number of Philosophers: ");
    scanf("%d", &n);
    pthread_t philosophers[n];
    int id[n];
    for (int i = 0; i < n; i++){
        sem_init(&chopstick[i], 0, 1);
```

```
    }
    for (int i = 0; i < n; i++){
        id[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &id[i]);
    }
    for (int i = 0; i < n; i++){
        pthread_join(philosophers[i], NULL);
    }
    for (int i = 0; i < n; i++){
        sem_destroy(&chopstick[i]);
    }
    return 0;
}
```

Output:

